Prince Mohammad Bin Fahd University
College of Computer Engineering and Science Secure Software Engineering

A Secure Software Engineering

Course Project / Spring 2025 – 2026

SECURE STUDENT INFORMATION SYSTEM

PROJECT Members

Julan Almuhawesh
ID: 202201147

Reyam Alhammadi
ID: 202201409

Muneerah Alsaeed
ID: 202101182

PROJECT OVERVIEW

OBJECTIVES

- Develop a robust and secure system for managing student information and academic records.
- Ensure data integrity and confidentiality throughout the student lifecycle.

CORE FUNCTIONALITY

- Secure User Authentication
- Role-Based Access Control
- Grade Management System
- Student Profile Management

TECH STACK

Java 21

Modern Backend Logic

Maven 3.9

Dependency Management

Apache Derby

Secure Database Storage

SECURITY REQUIREMENTS

FUNCTIONAL

- Secure User Authentication
- Role-Based Access Control (RBAC)
- Secure Grade Entry & Modification
- Profile Management with Validation

NON-FUNCTIONAL

- Confidentiality of student data
- Integrity of academic records
- Availability of the system
- Accountability through auditing

ASSET IDENTIFICATION

- Student Personal Records
- Instructor Credentials
- Official Academic Grades
- System Configuration Files

SECURITY GOALS

- Prevent Unauthorized Data Access
- Ensure 100% Data Accuracy
- Maintain 99.9% System Uptime
- Mitigate Top OWASP Vulnerabilities

THREAT MODELING (STRIDE)

S poofing

T ampering

R epudiation

I nfo Disclosure

D enial of Service

E levation of Privilege

KEY IDENTIFIED THREATS

THREAT SCENARIO	MITIGATION STRATEGY
SQL Injection Bypassing login or extracting data.	**Prepared Statements** & Parameterized Queries.
Grade Tampering Unauthorized modification of records.	**RBAC** & Strict Server-side Validation.
Brute-Force Credential guessing attacks.	**Account Lockout** & Rate Limiting.
Session Hijacking Stealing active user sessions.	**Secure Cookies** & HTTPS Encryption.

SECURE DESIGN STRATEGY

3-TIER ARCHITECTURE

Presentation level

Web Browser Interface (HTTPS, Input Validation, XSS Protection)

Application level

Java Logic Layer (Auth Enforcement, Session Management, API Security)

Data level

Apache Derby Database (Access Control, Encryption at Rest, Audit
Logs)

SECURITY PRINCIPLES

Least Privilege

Users and components get only the minimum access required.

Defense in Depth

Multiple layers of security controls provide redundancy.

Secure Defaults

System is secure out-of-the-box; access is denied by default.

Fail Securely

Errors result in a secure state without leaking information.

SECURE IMPLEMENTATION

CODING GUIDELINES

Input Validation
Strict regex-based server-side checks.

Output Encoding
Context-aware encoding to prevent XSS.

Error Handling
Generic messages to prevent info leaks.

Least Privilege
Minimal permissions for DB accounts.

SECURITY CONTROLS

Password Hashing
BCrypt Algorithm
Cost Factor: 12

SQL Protection
Prepared Statements
Parameterized Queries

Session Security
Secure & HttpOnly Cookies
Auto-Invalidation

DEFENSIVE CODE: SECURE LOGIN

IMPLEMENTATION LOGIC

```
public User authenticate(String user, String pass) {
  // 1. Input Validation
  if (!isValid(user) || !isValid(pass)) return null;

  // 2. SQL Injection Prevention
  String sql = "SELECT * FROM users WHERE username = ?";
  try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
  pstmt.setString(1, user);
    ResultSet rs = pstmt.executeQuery();

    // 3. Secure Password Verification
    if (rs.next() &&
      BCrypt.checkpw(pass,rs.getString("password"))) {
      resetAttempts(user);
      return mapUser(rs);
    }
    recordFailure(user);
  }
  return null;
}
```

SECURITY CONTROLS

Brute-Force Protection

Validation and secure authentication controls reduce repeated unauthorized login attempts.

Session Security

Secure ID generation and activity tracking.

Audit Logging

Detailed tracking of all authentication events.

SECURE TESTING & VERIFICATION

SECURITY TEST MATRIX

THREAT	TEST CASE DESCRIPTION	EXPECTED RESULT
SQL Injection	Inject malicious SQL into login fields (e.g., admin' OR '1'='1).	**PASS** Login fails; input treated as literal string.
XSS Attack	Inject script tags into profile fields or grade comments.	**PASS** Script is encoded or rejected; no execution.
IDOR	Modify student ID in URL to access other students' grades.	**PASS** Access denied; server verifies ownership.
Brute-Force	Perform 10 consecutive failed login attempts.	**PASS** Account locked for 15 minutes.

CONCLUSION & FUTURE WORK

KEY ACHIEVEMENTS

Robust Authentication

Secure login with BCrypt hashing and brute-force protection.

Effective Mitigation

Successful prevention of SQLi and XSS through secure coding.

Secure Architecture

Implementation of a resilient 3-tier defense-in-depth strategy.

THE ROAD AHEAD

Multi-Factor Authentication

Adding TOTP/SMS verification for enhanced security.

Advanced Auditing

Real-time monitoring and automated threat detection.

Web-Based Interface

Developing a full-scale UI using Spring Boot and Thymeleaf.

"Security is not a product, but a process of continuous vigilance and improvement."